

Auditoria Técnica Independente — ERPOS V2

Data: 2026-06-10 **Auditor:** Revisão externa (somente leitura — nenhum arquivo, função SQL ou dado foi alterado)
Projeto Supabase analisado: mdghhj emzdm euq p zuyzx **Escopo prioritário:** PDV/Caixa, Autoatendimento, QR Code/Mesa, Gestão de Pedidos, Relatórios, Estoque e configurações que impactam esses módulos. **Método:** leitura do código-fonte (frontend React + Edge Functions Deno), inspeção direta das funções SQL/RPC, enums, constraints e advisors de segurança do Postgres.

Este documento é independente da AUDITORIA_TECNICA_2026-06.md já existente. Onde houve sobreposição, os achados foram **reconfirmados na fonte**; onde divergi, está sinalizado. Vários achados **novos** (segurança de RPC, RLS) não constavam na auditoria anterior.

VEREDITO RÁPIDO

Eu **NÃO** colocaria este sistema em produção hoje. O núcleo de venda (criar pedido → pagar → baixar estoque → fechar caixa) funciona no caminho feliz, mas há:

- **3 falhas críticas de segurança** que permitem a qualquer pessoa com a chave pública (anon) **apagar todos os dados de qualquer loja, ler folha de pagamento de outros tenants e burlar isolamento multi-tenant** de vouchers.
- **Estorno do PDV Caixa 100% falso** (não toca no banco; senha 1234 hardcoded).
- **Registro de pagamento "não-bloqueante"** que pode marcar pedido como **pago sem nenhum pagamento gravado** → divergência de caixa e venda fantasma.
- **Pedido duplicado** por retry não-idempotente em falha de rede.
- **Estoque não devolvido** em vários caminhos de cancelamento e adicionais.

Detalhes, evidências e reprodução abaixo.

1. MAPA DA ARQUITETURA

1.1 Camadas



1.2 Módulos (escopo) → arquivos-chave

Módulo	Frontend	Backend
PDV Caixa	src/pages/pdv/caixa (53 arq.), PDVContext, SessaoContext	order-write, RPCs de pedido/pagamento/caixa
Autoatendimento	src/pages/autoatendimento, KioskAuthContext	kiosk-auth, order-write
Mesa QR	src/pages/mesa-qr, useMesaQRData	mesa-write, table-write
Mesa "cliente"/garçom	src/pages/mesa, MesasContext, MesaEdicaoContext	order-write, table-write
Gestão de Pedidos / KDS	src/pages/gestor-pedidos, src/pages/kds, KDSContext, ProducaoContext	order-write (update status)
Relatórios	src/pages/relatorios + hooks (useSalesReport, useCaixaReport, useCmv*, useDeliveryReport, useCancelamentosReport)	RPCs fn_get_*_report
Estoque	src/pages/estoque, EstoqueContext	stock-write, production-write

1.3 Tabelas compartilhadas (núcleo transacional)

- orders** — usada por TODOS os terminais e relatórios. PK `id`; UNIQUE (`session_id`, `number`). Enum `order_status` = {draft,new,preparing,ready,delivered,cancelled}.
- order_items** + **order_item_options** / **_observations** / **_units** / **_parts**.
- payments** — escrita por `fn_record_payment_bypass`; lida por caixa, relatórios, fechamento.
- sessions** (caixa lógico) + **cash_registers** (gaveta física) + **cash_movements** (sangria/suprimento).
- stock_movements** + **ingredients** — baixa por venda e restock.
- fin_cash_flow** — espelho financeiro de vendas/sangrias/estornos.

1.4 Funções compartilhadas (pontos únicos de falha)

- `fn_next_order_number(session_id, tenant_id)` — numeração (todos os terminais).
- `fn_create_order_bypass` / `fn_create_order_items_bypass` — criação (caixa, mesa, kiosk).
- `fn_record_payment_bypass` — todos os pagamentos.
- `deductStockForOrderItem()` / `buildDeductions()` (em `order-write`) — baixa de estoque de item/combo/adicional.
- `fn_close_cash_register_v2` — fechamento de caixa.

2. DIAGRAMAS LÓGICOS DOS FLUXOS

2.1 Pedido

Origem (Caixa/Garçom/Mesa QR/Kiosk/Delivery)
→ `useOrderSubmit.submitOrder` / `mesa-write.create_mesa_order`

→ fn_next_order_number (incrementa sessions.last_order_number)
→ fn_create_order_bypass (orders)
→ fn_create_order_items_bypass (order_items + options + obs)
→ [skip_kds] deductStockForOrderItem
→ print_queue (agente local imprime)
→ KDSContext lê orders/order_items (polling + realtime)
→ Gestão de Pedidos / KDS atualiza status
→ order-write.update_order_item_status → deriveOrderStatus → orders.status
→ Relatórios (fn_get_sales_report etc.)

2.2 Pagamento

PagamentoModal / PagamentoRapidoModal / autoatendimento
→ order-write.record_payment
→ fn_record_payment_bypass (payments) [■ retorna NULL se sem cash_register]
→ recáculo totalPaid → orders.is_paid
→ fin_cash_flow (income) + fin_income_routing → fn_bank_credit
→ fidelidade: customers.total_spent/points + loyalty_transactions
→ Fechamento: fn_close_cash_register_v2 (opening + cash payments + suprimentos - sangrias)
→ Relatórios de caixa (fn_get_cash_sessions_v2 / fallback direto)

2.3 Estoque

Venda: item ready/delivered → deductStockForOrderItem
→ buildDeductions (item_ingredients | combo_ingredients | combo_items → item_ingredients | options)
→ stock_movements (type=theoretical_out) + fn_update_ingredient_stock (clamp em 0)
Cancelamento: fn_cancel_and_refund_order + fn_restock_order (apenas ficha de item/combo; NÃO opções)
Movimentação manual / lote / inventário: stock-write → fn_add_stock_movement
Relatórios: fn_get_cmvm_report / useCmv*

2.4 Caixa

Abertura: SessaoContext.abrirCaixa → fn_open_cash_register (cash_registers status=open)
Vendas: payments.cash_register_id
Sangria/Suprimento: order-write.add_cash_movement → cash_movements + fin_cash_flow
Fechamento: fn_close_cash_register_v2 → expected = opening + Σ cash payments + Σ in - Σ out

2.5 Autoatendimento

Totem token → kiosk-auth (cria user 'tablet', JWT) → carrinho
→ order-write.create_order (origin=self_service)
→ busca cash_register aberto da sessão; SE existir → record_payment
→ SEM sessão/caixa → apenas addPedido() LOCAL (não grava no banco nem IndexedDB)

2.6 QR Code / Mesa

QR → mesa-write.lookup_mesa (cria table_session se caixa aberto)
→ create_participant → create_mesa_order (orders origin=table, destination=password)
→ print_queue
→ table-write.close_table_by_customer (valida itens entregues + conta paga)
Paralelamente, /mesa/:id (MesasContext) usa OUTRO modelo (destination_name livre) – DOIS fluxos divergentes

3. MAPA DE DEPENDÊNCIAS (quem depende de quê)

PDVContext ■■■ SessaoContext (sessão+caixa) ■■■ fn_open/close_cash_register
■ useOrderSubmit ■■■ order-write.create_order ■■■ fn_next_order_number
■ ■■ fn_create_order_bypass
■ ■■ fn_create_order_items_bypass
■ KDSContext (status, realtime) ■■■ order-write.update_order_item_status
■ printOrderQueue ■■■ print_queue ■■■ agente-local

PagamentoModal ■■■ order-write.record_payment ■■■ fn_record_payment_bypass ■■■ payments
■■■ fin_cash_flow / fn_bank_credit

CancelamentoModal ■■■ fn_cancel_and_refund_order + fn_restock_order (supabase.rpc direto)
EstornoModal ■■■ (NADA – só estado local) ■

Autoatendimento ■■■ KioskAuthContext ■■■ kiosk-auth ■■■ order-write (mesmo backend do caixa)
Mesa QR ■■■ useMesaQRData ■■■ mesa-write ■■■ fn_create_order_bypass (modelo password/participant)
Mesa cliente ■■■ MesasContext/MesaEdicaoContext ■■■ order-write + table-write (modelo destination_name)

Relatórios ■■■ fn_get_*_report (com fallback direto às tabelas no useCaixaReport)
Estoque ■■■ EstoqueContext ■■■ stock-write ■■■ fn_add_stock_movement / ingredient_batches

Pontos críticos para produção (concentração de risco): 1. `order-write` (arquivo único de 100 KB, 25+ ações) — qualquer exceção não tratada derruba criação/pagamento/cancelamento. 2. `fn_record_payment_bypass` (4 overloads) — toda receita passa aqui; falha silenciosa. 3. `sessions.last_order_number` — fonte da numeração; colisão de exibição entre sessões. 4. `deductStockForOrderItem` — toda a baixa de estoque.

4. MATRIZ DE RISCOS

Severidade	Bugs
CRÍTICO	BUG-01 (RPCs admin destrutivas abertas a anon), BUG-02 (RLS hr_payroll desligada), BUG-03 (RLS vouchers via user_metadata), BUG-04 (estorno PDV falso), BUG-05 (pagamento não-bloqueante → is_paid sem payment), BUG-06 (pedido duplicado por retry), BUG-07 (estoque não devolvido em adicionais/caminhos de cancelamento)
ALTO	BUG-08 (numeração duplicada entre sessões simultâneas), BUG-09 ("Em Rota" não persiste / sem enum), BUG-10 (autoatendimento offline não persiste pedido), BUG-11 (pagamento self-service sem caixa aberto → pedido não pago, cliente acha que pagou), BUG-12 (fin_cash_flow grava total do pedido por pagamento → receita inflada em split/multi-forma), BUG-13 (fallback do relatório de caixa zera formas/troco/descontos), BUG-14 (dois modelos divergentes de pedido de mesa), BUG-15 (fidelidade contada em duplicidade)
MÉDIO	BUG-16 (promoções avaliadas em UTC, não Brasília), BUG-17 (current_uses de promoção/cupom nunca incrementado → max_uses ineficaz), BUG-18 (restock sem checar se houve baixa → devolve estoque inexistente), BUG-19 (sem bloqueio de pedidos abertos no fechamento de caixa), BUG-20 (storage bucket menu-images permite listagem), BUG-21 (estoque negativo mascarado por clamp em 0), BUG-22 (comanda de cozinha sem fila/retry em alguns caminhos diretos)
BAIXO	BUG-23 (participant_name morto no mesa-write), BUG-24 (mesa cliente "fail-open" assume aberto em erro), BUG-25 (exec_sql exposto mas com guarda interna), BUG-26 (logs verbosos com PII no mesa-write)

5. LISTA DE BUGS (detalhada)

BUG-01 — RPCs administrativas destrutivas executáveis por `anon/authenticated` sem checagem de papel

- **Severidade:** CRÍTICA
- **Módulo:** Banco / Segurança (impacta tudo)
- **Tipo:** Segurança / Banco
- **Evidência técnica:** As funções `fn_admin_reset_tenant`, `fn_admin_delete_tenant`, `fn_admin_clear_orders`, `fn_admin_clear_stock` são SECURITY DEFINER e não têm nenhuma verificação interna de papel/permissão (corpo entra direto no DELETE). Verificado via `has_function_privilege: anon_exec = true` e `authenticated_exec = true` para todas. O corpo de `fn_admin_clear_orders` faz `DELETE FROM payments/orders/sessions/... WHERE tenant_id = p_tenant_id` recebendo o `tenant_id` por parâmetro, sem validar se o chamador pertence a esse tenant.

- **Como reproduzir:** Com a chave anon pública (embutida no bundle JS), enviar **POST** `/rest/v1/rpc/fn_admin_delete_tenant` com `{"p_tenant_id":"<qualquer-uuid>"}`. Nenhuma autenticação de admin é exigida.
- **Resultado esperado:** Apenas service_role ou admin-master deveria executar; o tenant do chamador deveria ser validado server-side.
- **Resultado encontrado:** Qualquer cliente apaga pedidos, pagamentos, estoque ou o tenant inteiro de **qualquer** loja.
- **Impacto:** Perda total de dados; vetor de sabotagem/ransomware. Bloqueador absoluto.
- **Sugestão:** `REVOKE EXECUTE ... FROM anon, authenticated`; exigir service_role; validar `p_tenant_id` contra `auth.uid()` + papel admin-master dentro da função.

BUG-02 — `hr_payroll` com RLS DESLIGADA (folha de pagamento exposta entre tenants)

- **Severidade:** CRÍTICA
- **Módulo:** Banco / Segurança (RH)
- **Tipo:** Segurança / Banco
- **Evidência técnica:** Advisor `policy_exists_rls_disabled` e `rls_disabled_in_public` nível **ERROR**: `"Table public.hr_payroll has RLS policies but RLS is not enabled"`. Existem políticas definidas (inclusive `hr_payroll_tenant_isolation`) mas, com RLS desabilitada, **todas são ignoradas**.
- **Como reproduzir:** Usuário autenticado de qualquer tenant faz `select * from hr_payroll` via PostgREST → retorna folha de todos os tenants.
- **Resultado esperado:** RLS habilitada com isolamento por tenant.
- **Resultado encontrado:** Leitura/escrita irrestrita de salários entre lojas.
- **Impacto:** Vazamento de dados sensíveis (LGPD), exposição salarial cross-tenant.
- **Sugestão:** `ALTER TABLE hr_payroll ENABLE ROW LEVEL SECURITY`; e revisar as políticas.

BUG-03 — RLS de `vouchers/voucher_transactions` baseada em `user_metadata` (burlável pelo usuário)

- **Severidade:** CRÍTICA
- **Módulo:** Banco / Segurança (Vouchers)
- **Tipo:** Segurança / Banco
- **Evidência técnica:** Advisor `rls_references_user_metadata` (ERROR): a política `vouchers_tenant_isolation` referencia `auth.jwt() -> user_metadata`. `user_metadata` é **editável pelo próprio usuário final** via `supabase.auth.updateUser`, então o isolamento de tenant pode ser forjado.
- **Como reproduzir:** Usuário altera o próprio `user_metadata.tenant_id` e passa a ler/gerar vouchers de outro tenant.
- **Resultado esperado:** Isolamento via tabela `user_tenants` (não-editável pelo usuário) ou `app_metadata`.
- **Impacto:** Fraude de vouchers/créditos entre lojas.
- **Sugestão:** Reescrever a policy usando `user_tenants/app_metadata`.

BUG-04 — Estorno do PDV Caixa é 100% simulado (não toca no banco; senha 1234 hardcoded)

- **Severidade:** CRÍTICA
- **Módulo:** PDV Caixa
- **Tipo:** Frontend / Regra de negócio
- **Evidência técnica:** `src/pages/pdv/caixa/components/EstornoModal.tsx:6` → `const SENHA_GERENTE MOCK = '1234'`. `handleConfirmar` (linha 49) compara a senha digitada com a constante e, em sucesso, apenas chama `onConfirmar(...)` e `setEtapa('sucesso')`. Em `PedidosRecentesPanel.tsx:1200` e `:1663`, `onConfirmar={() => {`

`setEstornado(true); setShowEstorno(false); }` — nenhuma chamada a RPC/edge function. Gera um "protocolo" fake `EST{Date.now()}` (linha 79).

- **Como reproduzir:** PDV Caixa → pedido pago → botão "Estornar" → senha `1234` → "Estorno Registrado". Recarregue a página: o badge "Estornado" some; `payments.is_refunded` continua `false`; `refunds` não tem registro; `fin_cash_flow` não tem estorno; auditoria do banco não registra.
- **Resultado esperado:** Estorno persistido (`refunds` + `payments.is_refunded` + `fin_cash_flow` + auditoria), com autorização real.
- **Resultado encontrado:** Operador acredita ter estornado; financeiro permanece intacto; dinheiro pode ser devolvido fisicamente ao cliente sem nenhum lastro contábil.
- **Impacto:** Perda financeira direta + caixa divergente + falsa sensação de auditoria.
- **Sugestão:** Substituir por fluxo real (o `CancelamentoModal` já chama `fn_cancel_and_refund_order`; ou usar `order-write.process_refund`). Remover senha hardcoded.

BUG-05 — `record_payment` "não-bloqueante" + `is_paid` somando valor não persistido → venda paga sem pagamento

- **Severidade:** CRÍTICA
- **Módulo:** PDV Caixa / Pagamento / Banco
- **Tipo:** Backend / Regra de negócio / Integridade
- **Evidência técnica:**
 - `fn_record_payment_bypass` retorna `NULL` e apenas emite `RAISE WARNING` quando não encontra `cash_register` aberto (verificado no `pg_get_functiondef`): "sem `cash_register_id` para order %, pulando insert em `payments`".
Nenhuma linha em `payments` é criada.
 - No frontend, todos os chamadores tratam erro como aviso: `PDVContext.tsx:437` (`console.warn(..., non-blocking)`), `PagamentoModal.tsx:310`, `autoatendimento` linha 497. O fluxo segue para a tela de sucesso.
 - Em `order-write` ação `record_payment` (linha 835-841): `totalPaid = Σ(payments não-refunded) + Number(amount)`. O `amount` é **sempre somado**, mesmo se o insert foi pulado (`NULL`). Se `totalPaid >= total` → `orders.is_paid = true`.
- **Como reproduzir:** Registrar pagamento de um pedido cujo `cash_register_id` resolvido é nulo (caixa físico fechado mas sessão lógica aberta; ou kiosk sem gaveta). Resultado: `orders.is_paid = true`, porém zero linhas em `payments`.
- **Resultado esperado:** Falha de pagamento deve bloquear a finalização; `is_paid` só quando há `payment` persistido.
- **Resultado encontrado:** Pedido marcado pago sem receita gravada → não entra no fechamento de caixa (que soma `payments`) nem nos relatórios → **divergência de caixa e venda fantasma**.
- **Impacto:** Perda financeira e divergência de fechamento.
- **Sugestão:** Tornar a falha bloqueante; só marcar `is_paid` quando `fn_record_payment_bypass` retornar id; calcular `totalPaid` apenas de `payments` reais.

BUG-06 — Pedido duplicado por retry não-idempotente em falha de rede

- **Severidade:** CRÍTICA
- **Módulo:** PDV / Autoatendimento / Mesa
- **Tipo:** Backend / Concorrência
- **Evidência técnica:** `useOrderSubmit.ts` envolve toda a chamada `create_order` em `retryAsync(..., maxRetries=3)` (linha 349). Cada tentativa chama `order-write.create_order`, que por sua vez chama `fn_next_order_number` (gera **novo** número) e `fn_create_order_bypass` (cria **novo** pedido). Não há chave de idempotência. A `UNIQUE (session_id, number)` **não** protege, pois cada retry usa número diferente.
- **Como reproduzir:** Criar pedido com rede instável: o backend cria o pedido, mas a resposta se perde (timeout). `retryAsync` re-tenta → segundo pedido idêntico com novo número. Dois tickets impressos, dois pedidos no KDS.

- **Resultado esperado:** Idempotência (client-token) — uma única ordem.
- **Resultado encontrado:** Pedido + comanda + (potencial) cobrança duplicados.
- **Impacto:** Produção duplicada, baixa de estoque dobrada, confusão no KDS, risco de cobrança dupla.
- **Sugestão:** Enviar `client_request_id` (UUID gerado uma vez por finalização) e usar `INSERT ... ON CONFLICT (client_request_id) DO NOTHING` em `orders`.

BUG-07 — Estoque de adicionais não é devolvido no cancelamento; caminhos de restock incompletos

- **Severidade:** CRÍTICA
- **Módulo:** Estoque / Cancelamento
- **Tipo:** Regra de negócio / Banco
- **Evidência técnica:** A baixa de venda (`buildDeductions` em `order-write`) inclui **opções/adicionais** (`buildOptionDeductions`, linha 185-209). Já o restock do banco `fn_restock_order` percorre apenas `item_ingredients`, `combo_ingredients` e `combo_items→item_ingredients` — **não há nenhuma devolução de `options.ingredient_id/production_recipe_id`**. Resultado: adicional dá baixa na venda mas nunca volta no cancelamento. Além disso, há **dois caminhos divergentes** de restock: `order-write.cancel_order` usa `restockForOrderItem` (que cobre options), enquanto `CancelamentoModal` chama `fn_restock_order` (que não cobre). O caminho usado pela UI é o `fn_restock_order` (incompleto).
- **Como reproduzir:** Vender hambúrguer + adicional "bacon" (com `ingredient_id`), cancelar com restock. Estoque de bacon **não** retorna.
- **Resultado esperado:** Restock simétrico à baixa (incluindo adicionais).
- **Resultado encontrado:** Estoque de adicionais drena permanentemente a cada cancelamento.
- **Impacto:** Estoque teórico diverge do real; CMV distorcido; rupturas indevidas.
- **Sugestão:** Unificar baixa e restock numa única função simétrica que cubra item, combo e opções.

BUG-08 — Numeração de pedido colide entre sessões/terminais simultâneos no mesmo dia

- **Severidade:** ALTA
- **Módulo:** Pedidos / Numeração
- **Tipo:** Banco / Regra de negócio

Evidência técnica: `fn_next_order_number` formata 'P' || DDMMYY || LPAD(`last_order_number`,4). O `last_order_number` é por **sessão** (`sessions.last_order_number`). A UNIQUE é (`session_id`, `number`). Se duas sessões de caixa do mesmo tenant estiverem abertas no mesmo dia (dois terminais), ambas começam em 0001 → **mesmo número visível P1006260001** em pedidos distintos (constraint não viola pois `session_id` difere).

Correção à auditoria anterior (BUG-016): não há risco de violação de constraint nem de duplicação **dentro** da mesma sessão (o `UPDATE ... RETURNING` é atômico). O risco real é **colisão de exibição** entre sessões concorrentes.

- **Como reproduzir:** Abrir dois caixas simultâneos; criar um pedido em cada. Ambos exibem o mesmo "#0001".
- **Resultado esperado:** Número único por tenant/dia.
- **Resultado encontrado:** Números repetidos → confusão na cozinha/retirada, ambiguidade em relatórios e comprovantes.
- **Impacto:** Operacional (entrega trocada) e rastreabilidade.
- **Sugestão:** Numeração por (`tenant_id`, `dia`) via sequência atômica dedicada, não por sessão.

BUG-09 — Status "Em Rota" não persiste (e não existe no enum do banco)

- **Severidade:** ALTA
- **Módulo:** Gestão de Pedidos / Delivery
- **Tipo:** Frontend / Integração
- **Evidência técnica:** `gestor-pedidos/page.tsx:344 handleEmRota` só faz `setPedidos(...)` (estado local) — nenhuma chamada a edge function. O enum `order_status` no banco é `{draft,new,preparing,ready,delivered,cancelled}` — **não inclui `em_rota`**. `STATUS_TO_DB` em `order-write` também não mapeia `em_rota`. `statusMappers.ts` trata `em_rota` apenas no frontend (mapeia para "Entregue"/rank 2-3).
- **Como reproduzir:** Marcar pedido de delivery como "Em Rota"; recarregar a página → status volta para "Pronto".
- **Resultado esperado:** Persistência do estado de entrega.
- **Resultado encontrado:** Estado efêmero; perdido em refresh, troca de terminal ou reabertura do KDS.
- **Impacto:** Perda de controle de entregas; entregador/pedido "somem" do quadro.
- **Sugestão:** Adicionar coluna/flag de delivery (ex.: `out_for_delivery_at`) ou estado próprio persistido; não reutilizar o enum de produção.

BUG-10 — Autoatendimento offline não persiste o pedido (perda total)

- **Severidade:** ALTA
- **Módulo:** Autoatendimento
- **Tipo:** Frontend / Integração
- **Evidência técnica:** `autoatendimento/page.tsx:505-540`: no ramo `else if (!sessionId)` ("Modo offline"), o código apenas monta `pedidoKDS` e chama `addPedido(pedidoKDS)` — estado React local. **Não** usa `saveOfflineOrder/IndexedDB` (ao contrário do PDV Caixa, que via `useOrderSubmit` cai em `saveOrderOffline`). Não há fila de sincronização.
- **Como reproduzir:** Totem sem sessão/conexão → cliente faz pedido → recarregar/fechar a aba → pedido desaparece (nunca foi ao banco nem ao IndexedDB).
- **Resultado esperado:** Persistir em IndexedDB e sincronizar ao reconectar (como o caixa).
- **Resultado encontrado:** Pedido perdido silenciosamente.
- **Impacto:** Venda perdida; cozinha pode ter produzido sem registro.
- **Sugestão:** Reusar `offlineDB/offlineSync` no kiosk.

BUG-11 — Pagamento self-service ignorado quando não há caixa aberto → cliente "paga" mas pedido fica não-pago

- **Severidade:** ALTA
- **Módulo:** Autoatendimento
- **Tipo:** Regra de negócio
- **Evidência técnica:** `autoatendimento/page.tsx:484` — `record_payment` só é chamado `if (cashRegisterId)`. Se não houver `cash_registers` aberto para a sessão, o pagamento é **silenciosamente pulado**; o pedido foi criado, mas sem pagamento e (dependendo do fluxo) sem `is_paid`. Combinado com BUG-05, se o caminho marcar `is_paid` mesmo assim, vira venda sem receita.
- **Como reproduzir:** Abrir sessão lógica sem gaveta (`cash_registers`) e operar totem com "pagar na hora".
- **Resultado esperado:** Bloquear venda self-service sem caixa, ou registrar a receita de outra forma.
- **Resultado encontrado:** Cliente conclui o fluxo achando que pagou; financeiro não registra.
- **Impacto:** Perda financeira / divergência.
- **Sugestão:** Exigir caixa aberto para self-service com pagamento, ou rota de pagamento independente de gaveta.

BUG-12 — `fin_cash_flow` grava o total do pedido por pagamento → receita inflada em split/múltiplas formas

- **Severidade:** ALTA
- **Módulo:** Financeiro / Relatórios
- **Tipo:** Backend / Regra de negócio
- **Evidência técnica:** `order-write` ação `record_payment`, linha 872: `saleAmount = orderTotalAmount > 0 ? orderTotalAmount : amount` e insere em `fin_cash_flow` com `amount = saleAmount` por **cada** chamada de `record_payment`. Quando um pedido é pago em **duas formas** (ex.: R\$30 dinheiro + R\$20 cartão), `record_payment` é chamado 2x; cada chamada grava o **total do pedido (R\$50)** em `fin_cash_flow` → R\$100 de receita para uma venda de R\$50. Há de-dup por `reference_id = paymentId`, mas como cada pagamento tem `paymentId` distinto, a de-dup não evita a soma dobrada.
- **Como reproduzir:** Finalizar um pedido com 2+ formas de pagamento; conferir `fin_cash_flow` (categoria "Vendas").
- **Resultado esperado:** Receita = total do pedido, uma vez (ou a soma dos `amount` reais).
- **Resultado encontrado:** Receita multiplicada pelo nº de formas de pagamento.
- **Impacto:** Faturamento/relatórios financeiros superestimados; DRE incorreto.
- **Sugestão:** Lançar em `fin_cash_flow` o `amount` real do pagamento (não o total do pedido), ou lançar o total uma única vez no fechamento do pedido.

BUG-13 — Fallback do relatório de caixa zera formas de pagamento, troco e descontos

- **Severidade:** ALTA
- **Módulo:** Relatórios (Caixa)
- **Tipo:** Frontend / Integridade
- **Evidência técnica:** `useCaixaReport.ts:131-235` — quando `fn_get_cash_sessions_v2` retorna vazio/erro, o fallback monta a sessão com `faturamento` (soma de orders) mas `por_forma_pagamento: []`, `total_troco: 0`, `total_descontos: 0`, `cash_transactions: []` (linhas 218-232).
- **Como reproduzir:** Forçar erro/timeout no RPC (carga alta de sábado) → relatório mostra faturamento mas "nenhuma forma de pagamento", troco zero, descontos zero.
- **Resultado esperado:** Fallback consistente ou estado de erro explícito.
- **Resultado encontrado:** Relatório internamente contraditório (tem faturamento, mas zero detalhamento) — leva a decisões erradas no fechamento.
- **Impacto:** Conferência de caixa enganosa.
- **Sugestão:** No fallback, montar formas de pagamento a partir de `payments/payment_methods` ou sinalizar "dados parciais".

BUG-14 — Dois modelos divergentes de pedido de mesa (QR vs cliente/garçom)

- **Severidade:** ALTA
- **Módulo:** Mesa / QR Code
- **Tipo:** Integração / Regra de negócio
- **Evidência técnica:** Mesa QR (`mesa-write.create_mesa_order`) grava `origin_type='table'`, `destination_type='password'`, `destination_name=access_token`, com `participant_id/table_session_id`. Já a Mesa "cliente"/garçom (`src/pages/mesa` via `order-write.create_order`) usa `destination_name` livre e modelo diferente. São caminhos paralelos que produzem registros estruturalmente distintos para o "mesmo" conceito.
- **Como reproduzir:** Comparar pedidos criados pelos dois fluxos na tabela `orders`.
- **Resultado esperado:** Um único modelo de pedido de mesa.

- **Resultado encontrado:** Inconsistência de dados; relatórios e fechamento de mesa precisam tratar dois formatos; risco de pedidos órfãos de `table_session`.
- **Impacto:** Manutenção, fechamento de mesa e relatórios divergentes.
- **Sugestão:** Convergir para um único modelo canônico de pedido de mesa.

BUG-15 — Fidelidade do cliente contabilizada em duplicidade

- **Severidade:** ALTA
- **Módulo:** Clientes / Fidelidade
- **Tipo:** Regra de negócio / Banco
- **Evidência técnica:** Em `order-write`, o gasto/pontos do cliente é incrementado em **dois lugares**: (a) `record_payment` (linha 854-859) faz `customers.total_spent += orderAmount + loyalty_transactions`; (b) `update_order_item_status` quando entrega (`delivered`) chama `fn_update_customer_spent(customer_id, total_amount)` (linha 727), que **também** soma `total_spent` e `visit_count`. Um pedido pago e entregue conta o gasto duas vezes.
- **Como reproduzir:** Pedido com `customer_id`, pagar e marcar entregue; conferir `customers.total_spent/visit_count`.
- **Resultado esperado:** Gasto/visita contados uma vez.
- **Resultado encontrado:** `total_spent`, `visit_count` e `loyalty_points` inflados.
- **Impacto:** Tier de fidelidade incorreto, métricas de cliente erradas.
- **Sugestão:** Centralizar a atualização de fidelidade em um único gatilho (pagamento OU entrega, não ambos).

BUG-16 — Promoções/combo avaliados em UTC, não em Brasília

- **Severidade:** MÉDIA
- **Módulo:** Promoções / Pedido
- **Tipo:** Regra de negócio
- **Evidência técnica:** `order-write` linhas 94-97: `nowTimeMinutes()` usa `getUTCHours/getUTCMinutes`, `todayDayOfWeek()` usa `getUTCDay()`, `todayDateStr()` usa `toISOString()`. `isRuleActiveNow` (linha 134) compara `time_from/time_until/days_of_week` contra esses valores UTC. Para o Brasil (UTC-3), a janela da promoção desloca 3h e o dia da semana vira à meia-noite UTC (21h local).
- **Como reproduzir:** Promoção "happy hour 18:00-20:00" só será aplicada às 21:00-23:00 locais; promoção de "segunda-feira" muda às 21h de domingo.
- **Resultado esperado:** Avaliar em America/Sao_Paulo (como `fn_next_order_number` já faz).
- **Resultado encontrado:** Promoções aplicadas/expiradas em horário errado.
- **Impacto:** Descontos indevidos ou negados; perda de receita ou reclamação.
- **Sugestão:** Calcular hora/dia/data em Brasília no `order-write`.

BUG-17 — `current_uses` de promoção/cupom nunca é incrementado (limite de uso ineficaz)

- **Severidade:** MÉDIA
- **Módulo:** Promoções
- **Tipo:** Regra de negócio
- **Evidência técnica:** `isRuleActiveNow` (linha 144) bloqueia quando `current_uses >= max_uses_total`, mas em nenhum ponto de `order-write` (nem em `apply_promotions`, nem em `create_order`) há `UPDATE promotion_rules SET current_uses = current_uses + 1`. `update_promotion_rule` inclusive remove `current_uses` do payload (linha 440).

- **Como reproduzir:** Criar cupom com `max_uses_total=1`; usar várias vezes — todas passam.
- **Resultado esperado:** Incremento atômico do uso a cada aplicação.
- **Resultado encontrado:** Limite de uso total nunca é atingido.
- **Impacto:** Abuso de cupom/promoção; perda de receita.
- **Sugestão:** Incrementar `current_uses` ao confirmar o pedido com a promoção aplicada.

BUG-18 — Restock não verifica se houve baixa → devolve estoque que nunca saiu

- **Severidade:** MÉDIA
- **Módulo:** Estoque / Cancelamento
- **Tipo:** Regra de negócio
- **Evidência técnica:** A baixa só ocorre quando o item chega a `ready/delivered` (`update_order_item_status` linha 706; ou `skip_kds` na criação). Mas `fn_restock_order/restockForOrderItem` devolvem estoque para **todo** item cancelado, **sem checar** se a baixa chegou a acontecer. Cancelar um pedido `new` (que nunca baixou) com restock → cria `stock_movements type='in'` e incrementa estoque indevidamente.
- **Como reproduzir:** Criar pedido (status `new`, sem baixa) → cancelar com `restock=true` → estoque do insumo aumenta.
- **Resultado esperado:** Só devolver o que efetivamente saiu (casar com `stock_movements theoretical_out` do pedido).
- **Resultado encontrado:** Estoque inflado em cancelamentos de pedidos não preparados.
- **Impacto:** Estoque teórico acima do real; CMV distorcido.
- **Sugestão:** Restock baseado nos `stock_movements` de baixa reais do pedido, não na ficha técnica.

BUG-19 — Fechamento de caixa não bloqueia/alerta pedidos abertos

- **Severidade:** MÉDIA
- **Módulo:** Caixa
- **Tipo:** Regra de negócio
- **Evidência técnica:** `close_cash_register/fn_close_cash_register_v2` calculam o esperado e fecham sem verificar pedidos `new/preparing/não-pagos` da sessão. (Há `check-session-pending` como função separada, mas o fechamento de caixa não a impõe.)
- **Como reproduzir:** Fechar caixa com pedido em preparo/não pago aberto → fecha normalmente.
- **Resultado esperado:** Bloquear ou alertar sobre pendências.
- **Resultado encontrado:** Caixa fecha com pendências; receita/itens podem ficar órfãos da sessão.
- **Impacto:** Divergência de fechamento; pedidos sem caixa.
- **Sugestão:** Validar pendências antes de permitir o fechamento.

BUG-20 — Bucket `menu-images` permite listagem pública de todos os arquivos

- **Severidade:** MÉDIA
- **Módulo:** Storage / Configuração
- **Tipo:** Segurança
- **Evidência técnica:** Advisor `public_bucket_allows_listing`: *"Public bucket `menu-images` has 1 broad SELECT policy ... allowing clients to list all files."* Acesso por URL de objeto não exige policy de listagem.
- **Impacto:** Enumeração de imagens de todos os tenants.
- **Sugestão:** Remover a policy de SELECT ampla; manter apenas acesso por URL direta.

BUG-21 — Estoque negativo mascarado por `GREATEST(0, ...)` (consumo "invisível")

- **Severidade:** MÉDIA
- **Módulo:** Estoque
- **Tipo:** Regra de negócio
- **Evidência técnica:** `fn_update_ingredient_stock` e `fn_add_stock_movement` aplicam `current_stock = GREATEST(0, current_stock + delta)`. Quando o estoque real já estava em 0, vendas seguem ocorrendo (não há bloqueio — apenas alerta visual via `fn_check_stock_alert_for_items`), e o consumo além de zero é descartado silenciosamente.
- **Como reproduzir:** Zerar um insumo e continuar vendendo itens que o consomem; o estoque fica preso em 0 e o consumo real não é refletido.
- **Resultado esperado:** Permitir negativo controlado (para visibilidade) ou bloquear venda.
- **Resultado encontrado:** Divergência teórico×real oculta; CMV subestimado.
- **Impacto:** Inventário e CMV imprecisos.
- **Sugestão:** Registrar o consumo real e expor o negativo, com política explícita de bloqueio/alerta.

BUG-22 — Comanda de cozinha em caminhos diretos sem fila/retry

- **Severidade:** MÉDIA
- **Módulo:** Impressão
- **Tipo:** Integração
- **Evidência técnica:** `useOrderSubmit/mesa-write` enfileiram em `print_queue` (com retry via agente). Porém há caminhos de impressão direta (`sendToPrinter/printKitchenTicket` no frontend) sem fila nem retry — se a impressora/agente estiver offline, o ticket é perdido silenciosamente. (Coerente com BUG-028 da auditoria anterior.)
- **Impacto:** Pedido sem comanda na cozinha → item não produzido.
- **Sugestão:** Padronizar toda impressão pela `print_queue`.

BUG-23 — `participant_name` enviado mas ignorado no `mesa-write`

- **Severidade:** BAIXA · **Tipo:** Integração. Campo morto no payload de `create_mesa_order` (não persiste o nome do participante). Coerente com BUG-020 anterior.

BUG-24 — Mesa cliente "fail-open" assume sessão aberta em erro de fetch

- **Severidade:** BAIXA · **Tipo:** Regra de negócio. `MesaClientePage` assume 'open' em falha de checagem de status; permite pedido em estabelecimento fechado. Coerente com BUG-024 anterior.

BUG-25 — `exec_sql(text)` exposto a anon/authenticated (mitigado por guarda interna)

- **Severidade:** BAIXA (defesa em profundidade) · **Tipo:** Segurança. `EXECUTE` concedido a anon/authenticated, mas o corpo bloqueia `current_user NOT IN (service_role, supabase_admin, postgres)`. A guarda funciona, porém a exposição não deveria existir. **Sugestão:** `REVOKE EXECUTE FROM anon, authenticated`.

BUG-26 — Logs verbosos com dados de pedido/PII no `mesa-write`

• **Severidade:** BAIXA · **Tipo:** Segurança/Operacional. `console.log` extensivo de payloads (itens, estações, tokens) em produção. **Sugestão:** reduzir verbosidade e nunca logar `access_token`.

6. SIMULAÇÃO OPERACIONAL — Sábado à noite, 50 clientes

Cenário	Resultado	Bug
Duplo clique em "Finalizar"	Protegido por <code>submittingRef</code> em <code>useOrderSubmit</code>	OK
Finalizar com rede instável (timeout)	Pedido duplicado (retry recria)	BUG-06
Caixa físico fechado, sessão aberta	Pagamento perdido, pedido marcado pago	BUG-05
Pagamento em 2 formas (dinheiro+cartão)	Receita dobrada em <code>fin_cash_flow</code>	BUG-12
Dois terminais de caixa simultâneos	Números de pedido repetidos	BUG-08
Estorno no PDV após cobrança	Estorno falso; dinheiro sai sem lastro	BUG-04
Cancelar pedido com adicional (restock)	Adicional não volta ao estoque	BUG-07
Cancelar pedido <code>new</code> com restock	Estoque inflado (nunca baixou)	BUG-18
Delivery → "Em Rota" e refresh	Status some	BUG-09
Totem sem conexão/sessão	Pedido perdido	BUG-10
Insumo zerado, vendas continuam	Consumo real escondido (clamp 0)	BUG-21
Pedido pago + entregue (cliente cadastrado)	Fidelidade contada 2x	BUG-15
Happy hour 18-20h	Aplica 21-23h (UTC)	BUG-16
Cupom <code>max_uses_total=1</code>	Usável infinitas vezes	BUG-17
Relatório de caixa sob carga (RPC falha)	Faturamento sem detalhamento (zeros)	BUG-13
KDS sob carga	Status nunca regride (lógica de rank robusta)	OK

7. RISCOS PARA PRODUÇÃO (bloqueadores obrigatórios antes de receber clientes reais)

Inegociáveis (corrigir antes de qualquer piloto real): 1. **BUG-01** — fechar RPCs admin destrutivas (anon pode apagar tudo). 2. **BUG-02 / BUG-03** — habilitar RLS em `hr_payroll`; corrigir RLS de vouchers (não usar `user_metadata`). 3. **BUG-04** — substituir o estorno falso por fluxo real. 4. **BUG-05 / BUG-11** — pagamento deve ser bloqueante; `is_paid` só com `payment` persistido. 5. **BUG-06** — idempotência na criação de pedido. 6. **BUG-07 / BUG-18** — restock simétrico e baseado em baixa real.

Fortemente recomendados antes de escalar: 7. BUG-12 (receita inflada em split), BUG-13 (relatório de caixa), BUG-08 (numeração), BUG-09 (Em Rota), BUG-10 (kiosk offline), BUG-15 (fidelidade), BUG-14 (unificar modelo de mesa).

Higiene de segurança: BUG-20 (bucket), BUG-25 (`exec_sql`), BUG-26 (logs/PII), além dos ~494 *advisors* WARN (`search_path` mutável, SECURITY DEFINER amplos) que devem ser triados.

8. CONCLUSÃO

"Eu colocaria este sistema em produção hoje?" — Não.

Justificativa técnica: A camada de venda no caminho feliz é competente — proteção de duplo clique, fila de impressão com retry, derivação de status do KDS sem regressão, validação de consistência financeira na criação do pedido ($\text{subtotal} - \text{desconto} + \text{taxa} = \text{total}$). Mas o sistema falha exatamente onde o dinheiro e o estoque mudam de mãos sob condições reais:

- **Segurança de banco inaceitável:** funções administrativas que apagam dados estão abertas à chave pública (BUG-01); folha de pagamento e vouchers vazam entre tenants (BUG-02/03). Qualquer um desses três, isoladamente, já reprovava o sistema para produção.
- **Integridade financeira frágil:** estorno falso (BUG-04), pagamento que marca venda como paga sem gravar receita (BUG-05), receita dobrada em split (BUG-12) e relatório de caixa que zera o detalhamento sob carga (BUG-13) tornam o fechamento de caixa não-confiável — o oposto do que um PDV precisa garantir.
- **Consistência de pedido e estoque:** pedido duplicado por retry (BUG-06), estoque de adicionais que nunca volta (BUG-07) e restock que infla estoque (BUG-18) corrompem inventário e produção num sábado movimentado.

O sistema está num estágio de **"demo avançada que funciona no roteiro feliz"**, não de produto pronto para operar com dinheiro real. Com a correção dos 6 blocos inegociáveis da Seção 7 (estimo o maior esforço em BUG-05/06/07, que exigem refatorar o ciclo de pagamento/estoque com idempotência e simetria), seguida de um teste de carga focado nos cenários da Seção 6, o sistema pode chegar a um piloto controlado.

Recomendação: congelar o go-live; tratar Seção 7 itens 1-6; reauditar pagamento/estoque/cancelamento end-to-end; só então piloto com uma loja e acompanhamento diário de divergência de caixa e inventário.

9. ANÁLISE DE LÓGICA DE NEGÓCIO E USABILIDADE

Esta seção avalia se a *lógica* do sistema faz sentido para o uso real (cálculo de caixa, baixa de estoque, produção, CMV, inventário), independentemente de bugs de código.

9.1 Diferença de caixa — base correta, 2 furos

A fórmula de `fn_close_cash_register_v2` ($\text{esperado} = \text{abertura} + \sum \text{pagamentos em dinheiro} + \sum \text{suprimentos} - \sum \text{sangrias}$; $\text{diferença} = \text{contado} - \text{esperado}$) está **correta**: só conta `type='cash'`, e o troco já é abatido porque `payments.amount` guarda o valor líquido retido ([PagamentoModal.tsx:268](#)). Furos: ver **BUG-27**.

9.2 Baixa de estoque — variáveis certas, gatilho frágil

A baixa (`buildDeductions`, [order-write/index.ts:211](#)) usa `item_ingredients`, `combo_ingredients/combo_items`, e opções — modelagem correta. O problema é o **gatilho** e a **modelagem**: ver **BUG-28**, **BUG-33**.

9.3 Produção — conceito certo, 2 armadilhas

`fn_register_production_and_stock_v2` consome insumos crus (`manual_out`) e dá entrada num insumo "produto acabado" (`in`). O modelo só fecha se a ficha do prato apontar o insumo **produzido**. Ver **BUG-29** (duplo consumo) e **BUG-30** (custo não vai ao CMV).

9.4 CMV — subconta sistematicamente

`fn_get_cm_v_report` custeia só a partir de `item_ingredients` do item de menu. Ver **BUG-32** (combos/adicionais = custo 0) e **BUG-34** (preço recalculado retroativamente).

9.5 Inventário

`confirm_inventory` sobrescreve `current_stock` com o contado (correto), mas o movimento de ajuste perde o sinal. Ver **BUG-31**.

BUG-27 — Reembolso parcial em dinheiro não abate o caixa esperado

- **Severidade:** ALTA · **Módulo:** Caixa/Financeiro · **Tipo:** Regra de negócio
- **Evidência:** `register_partial_refund` ([order-write/index.ts:358](#)) grava `fin_cash_flow` (despesa) mas **não cria `cash_movements` nem marca `is_refunded`**. `fn_close_cash_register_v2` só lê `cash_movements` e `payments` → o dinheiro devolvido fisicamente não é abatido do esperado.
- **Reprodução:** Reembolso parcial em dinheiro → fechar caixa → gaveta acusa falta equivalente ao reembolso.
- **Impacto:** Divergência sistemática de caixa em reembolsos parciais.
- **Sugestão:** Todo reembolso em dinheiro deveria gerar uma sangria (`cash_movements type='out'`).

BUG-28 — Baixa de estoque depende de marcação manual no KDS

- **Severidade:** ALTA · **Módulo:** Estoque · **Tipo:** Regra de negócio / Usabilidade
- **Evidência:** A baixa (`theoretical_out`) só ocorre quando o item vira `ready/delivered` ([order-write/index.ts:706, 757](#)); exceção: `skip_kds` baixa na criação. Não há baixa no momento do pagamento/venda.
- **Reprodução:** Operar sem marcar itens como prontos/entregues no KDS → estoque teórico nunca decrementa.
- **Impacto:** Em operação que não usa o KDS com disciplina, o controle de estoque/CMV vira ficção, sem nenhum alerta.
- **Sugestão:** Permitir baixa no pagamento (ou alternativa configurável), desacoplando o estoque da disciplina do KDS.

BUG-29 — Duplo consumo de estoque quando a ficha aponta insumos crus em vez do produzido

- **Severidade:** ALTA · **Módulo:** Estoque/Produção · **Tipo:** Regra de negócio
- **Evidência:** A produção consome crus e gera o insumo "produto acabado". Se a ficha do item de menu listar os **crus** (e não o produzido), a venda baixa os crus **de novo** e o estoque do produzido nunca é consumido. Nada no sistema detecta/avisa a incoerência.
- **Reprodução:** Produzir "molho" em batch e cadastrar a ficha do lanche com tomate/cebola crus → crus baixam 2× (produção + venda); estoque de "molho" só cresce.
- **Impacto:** Estoque cru drena em dobro; produto acabado acumula; CMV e inventário divergem.
- **Sugestão:** Convenção/validação que impeça ficha de prato referenciar crus que já são insumo de uma receita de produção.

BUG-30 — Custo do insumo produzido não é propagado ao CMV

- **Severidade:** MÉDIA · **Módulo:** Produção/CMV · **Tipo:** Regra de negócio
- **Evidência:** `fn_register_production_and_stock_v2` calcula `unit_cost` do batch mas **não atualiza `ingredients.unit_price`** do produto acabado. O CMV lê `ingredients.unit_price`.
- **Impacto:** Itens que usam insumo produzido têm CMV subestimado se o preço do produzido não for setado à mão.
- **Sugestão:** Atualizar `unit_price` do insumo de saída com o custo unitário da produção.

BUG-31 — Ajuste de inventário não carrega o sinal (perda registrada como entrada)

- **Severidade:** MÉDIA · **Módulo:** Estoque/Inventário · **Tipo:** Regra de negócio
- **Evidência:** `confirm_inventory` ([stock-write/index.ts:294](#)) cria movimento `inventory_adjustment` com `quantity = abs(diferenca)`. Em `fn_add_stock_movement`, `inventory_adjustment` **não** está no conjunto de saídas → é sempre +. O `current_stock` fica certo (há `UPDATE` direto), mas o livro de movimentações registra toda quebra como acréscimo.
- **Impacto:** Relatórios que somam movimentos divergem; rastro de perdas de inventário fica invertido.
- **Sugestão:** Gerar o ajuste com o sinal real da diferença (entrada vs saída).

BUG-32 — CMV ignora combos e adicionais

- **Severidade:** ALTA · **Módulo:** Relatórios/CMV · **Tipo:** Regra de negócio
- **Evidência:** `fn_get_cmv_report` faz `LEFT JOIN menu_items mi ON mi.id = oi.item_id`. Combo tem `item_id` nulo → custo 0. O custo de opções/adicionais não é incluído.
- **Impacto:** CMV global subestimado e margem inflada em qualquer operação com combos/adicionais.
- **Sugestão:** Custear combos (via `combo_ingredients/combo_items`) e somar o custo dos adicionais.

BUG-33 — Combo com ingredientes diretos + sub-itens baixa só os diretos

- **Severidade:** MÉDIA · **Módulo:** Estoque · **Tipo:** Regra de negócio
- **Evidência:** `buildDeductions` ([order-write/index.ts:217-227](#)) usa `combo_ingredients` ou `combo_items` (exclusivo). Combo modelado com os dois perde a baixa dos sub-itens.
- **Sugestão:** Somar as duas fontes em vez de tratá-las como exclusivas.

BUG-34 — CMV histórico recalculado com o preço atual do insumo

- **Severidade:** MÉDIA · **Módulo:** Relatórios/CMV · **Tipo:** Regra de negócio
- **Evidência:** O custo usa `ingredients.unit_price` "ao vivo"; não há snapshot do custo no momento da venda. Mudar o preço do insumo altera o CMV de períodos passados.
- **Sugestão:** Gravar o custo unitário no `order_items` no momento da venda (snapshot) e usá-lo no CMV.

10. DIAGNÓSTICO — Pedidos que não registram mudança de fase

Investigação do sintoma relatado: *"terminei o dia com pedidos em preparo / a entregar que eu já tinha sinalizado pronto/entregue"*. Os pedidos não persistem a mudança de fase. **Causa raiz confirmada na fonte.**

10.1 O que acontece (encadeamento)

1. No KDS, ao avançar a fase, o estado **local** é atualizado de forma otimista — o card mostra "Pronto"/"Entregue" na hora ([kds/page.tsx:181, 250](#)).
2. A gravação no banco é feita por `updateItemStatusRemote` / `updateUnitStatusRemote` / `updatePartStatusRemote`, chamadas **sem `await` e `fire-and-forget`** ([kds/page.tsx:235-239, 284-286](#)).
3. Dentro dessas funções, qualquer erro é **apenas logado** (`console.error`) — sem retry, sem fila offline, sem aviso ao operador ([KDSContext.tsx:1329, 1379, 1422](#)).
4. O merge de realtime usa a regra **"nunca regredir"** (`DB_STATUS_RANK2`, [KDSContext.tsx:898-915](#)): mantém o status local mais avançado mesmo quando o banco discorda. Isso **mascara** a falha — o operador continua vendo "Entregue" o

dia todo.

5. No fim do dia (carregamento novo / outro terminal / relatório), o estado é lido do banco → ainda "preparo"/"a entregar". É exatamente o sintoma relatado.

10.2 Gatilhos que disparam a falha

- **Qualquer instabilidade de rede no momento do toque** (Wi-Fi da cozinha oscilando, edge function lenta/timeout, 500 transitório). Não há `navigator.onLine` nem fila offline para status — diferente da criação de pedido, que tem `offlineDB`. Um blip = fase perdida.
- **Pedido travado em edição (`is_editing`)** — ver BUG-37: o update do KDS volta 423 e é silenciosamente descartado.
- **Atualização por item, não transacional** — `handleAvancarPedido` dispara N chamadas separadas ([kds/page.tsx:236](#)); falha parcial deixa o pedido com status misto (alguns itens avançam, outros não).
- **Ações "Em Rota"/"Entregar" do Gestor de Pedidos** que nunca persistem (BUG-09/BUG-38).

BUG-35 — Mudança de fase é fire-and-forget com erro engolido (sem retry/fila/aviso)

- **Severidade:** CRÍTICA (operacional) · **Módulo:** KDS/Gestão de Pedidos · **Tipo:** Frontend / Integração
- **Evidência:** [kds/page.tsx:235-239](#) chama `updateItemStatusRemote(...)` sem `await`; a função em [KDSContext.tsx:1300-1332](#) trata erro só com `console.error` e comenta *"Realtime triggers reload automatically"* — mas se a escrita falhou, **não há nada no banco para o realtime propagar**. Nenhuma checagem de `navigator.onLine`, nenhuma fila de re-tentativa.
- **Reprodução:** Marcar item como pronto/entregue com a rede oscilando por 1-2 s → UI mostra avançado; `order_items.status` no banco permanece no estado anterior; nenhum erro é exibido.
- **Resultado esperado:** Gravação confirmada (com retry/fila offline) ou aviso visível de falha.
- **Resultado encontrado:** Mudança de fase perdida silenciosamente; só aparece no fim do dia/outro terminal.
- **Impacto:** Pedidos "presos" em preparo/a entregar no fechamento; SLA e relatórios de produção errados; risco de retrabalho/entrega duplicada.
- **Sugestão:** Aguardar a confirmação, com retry e fila offline (como em `useOrderSubmit`), e sinalizar falha ao operador.

BUG-36 — Merge "nunca regredir" mascara a falha de gravação

- **Severidade:** ALTA · **Módulo:** KDS · **Tipo:** Frontend
- **Evidência:** [KDSContext.tsx:898-915](#) — `DB_STATUS_RANK2` preserva o status local quando `prevRank > mergedRank`, ou seja, ignora o que o banco diz quando o local está "mais à frente". Combinado com BUG-35, o operador nunca percebe que a gravação não aconteceu.
- **Impacto:** Falha invisível durante todo o turno; diagnóstico tardio.
- **Sugestão:** Só preservar o status local enquanto houver gravação **confirmada** pendente; caso contrário, deixar o banco ser a verdade.

BUG-37 — Lock de edição sem expiração trava o avanço de fase indefinidamente

- **Severidade:** ALTA · **Módulo:** Pedidos/KDS · **Tipo:** Backend / Regra de negócio
- **Evidência:** `order-edit-lock.start` ([order-edit-lock/index.ts:67-114](#)) grava `is_editing=true` e `editing_started_at`, mas **nada libera o lock automaticamente** — `editing_started_at` nunca é usado para expiração. Se o PDV abre a edição e fecha a aba/cai/perde rede antes do `finish`, o pedido fica travado. `checkOrderLock` ([order-write/index.ts:289](#)) então rejeita os updates do KDS com **423**, e o handler do KDS apenas marca `isEditing` e descarta a ação ([KDSContext.tsx:1315-1327](#)).
- **Reprodução:** No PDV, iniciar edição de um pedido e fechar a aba sem concluir → no KDS, tentar avançar a fase desse pedido → 423, mudança descartada; o pedido fica preso até `force_unlock` (só admin).

- **Impacto:** Pedidos que nunca avançam de fase; precisam de intervenção manual de admin.
- **Sugestão:** Auto-expirar o lock por `editing_started_at` (ex.: 2-5 min) no `checkOrderLock`; e, no KDS, re-tentar após o desbloqueio.

BUG-38 — "Em Rota" e "Entregar" do Gestor de Pedidos não persistem

- **Severidade:** ALTA · **Módulo:** Gestão de Pedidos/Delivery · **Tipo:** Frontend (ver também BUG-09)
- **Evidência:** `gestor-pedidos/page.tsx:344` `handleEmRota` e o `handleEntregar` adjacente fazem só `setPedidos(...)` (estado local), sem chamada ao backend. `em_rota` sequer existe no enum `order_status`.
- **Impacto:** Pedidos de delivery marcados "Em Rota"/"Entregue" pelo Gestor voltam ao estado anterior no fim do dia/refresh.
- **Sugestão:** Persistir via `order-write` (e modelar o estado de entrega no banco).

Atualização da matriz de riscos (Seções 9-10)

Severidade	Bugs adicionais
CRÍTICO	BUG-35 (mudança de fase fire-and-forget)
ALTO	BUG-27, BUG-28, BUG-29, BUG-32, BUG-36, BUG-37, BUG-38
MÉDIO	BUG-30, BUG-31, BUG-33, BUG-34

Sobre o sintoma relatado (pedidos presos no fechamento): a combinação **BUG-35 + BUG-36 + BUG-37** é a explicação direta — a gravação da fase pode falhar sem aviso, o estado local mascara a falha durante o turno, e pedidos travados em edição nem conseguem avançar. São correções prioritárias junto com os bloqueadores da Seção 7.

11. ANÁLISE DO MÓDULO FINANCEIRO

O módulo é **rico em funcionalidades** (17 abas: Visão Geral, Receitas, Despesas, Fluxo de Caixa, Previsão, Contas a Pagar/Receber, Orçamentos, Compras, RH/Folha, Centro de Custos, DRE, Conciliação bancária, Bancos, Implantação) e tem boas intenções contábeis (DRE com modos **Caixa** e **Competência**, antecipação de recebíveis com dedup, recorrência de contas, roteamento de receita para bancos, conciliação Stone/OFX, aging). O problema **não é falta de recurso — é a falta de um livro-razão único**: cada tela recalcula a receita a partir de uma tabela operacional diferente, com filtros e regimes diferentes, então **as telas não reconciliam entre si**.

11.1 Fluxo de dados (de onde vem → pra onde vai)

```
VENDA (PDV)
■ payments (sempre, no momento da venda – inclui cartão a prazo)
■ fin_cash_flow (income, origin=auto_sale) ■ só se days_to_receive = 0
■ fin_receivable_installments (pending) ■ só se days_to_receive > 0
■ ■ receive_installment → fin_cash_flow (income, auto_sale) [na data do recebimento]
■ fn_bank_credit (se houver roteamento configurado)

DESPESA
■ fin_accounts_payable → pay_bill → fin_cash_flow (expense) + fn_bank_debit
■ fin_purchases / fin_purchase_items → contas a pagar
■ hr_payroll → pay_payroll → fin_cash_flow (expense)

LEITURA (cada tela usa uma fonte diferente):
Visão Geral / Fluxo de Caixa → receita de ORDERS + despesa de FIN_CASH_FLOW
DRE "Caixa" → receita de PAYMENTS
DRE "Competência" → receita de PAYMENTS + FIN_RECEIVABLE_INSTALLMENTS
```

A receita existe em **três fontes** (`orders`, `payments`, `fin_cash_flow`) e cada tela lê de uma. Como cada fonte tem um conjunto/filtro diferente (pedido pago vs não pago, recebido vs a receber, com/sem estorno), o **faturamento do mesmo**

período difere de tela para tela.

11.2 Tabela comparativa — por que os números não batem

Tela	Receita lida de	Despesa lida de	Conta pedido não pago?	Cartão a prazo
Visão Geral	orders.total_amount	fin_cash_flow (saídas)	Sim (infla)	conta na venda
Fluxo de Caixa	orders.total_amount	fin_cash_flow (saídas)	Sim	conta na venda
DRE "Caixa"	payments.amount	pagas + compras + folha	Não	conta na venda (não no recebimento)
DRE "Competência"	payments + recebíveis	pendentes + pagas	Não	conta 2x

Para a **mesma** semana, essas quatro telas produzem quatro receitas diferentes. Isso é um problema de credibilidade do módulo inteiro: o gestor não sabe qual número é "o certo".

11.3 O que está bem feito (crédito devido)

- Separação explícita **Caixa × Competência** no DRE (intenção correta).
- `fin_cash_flow` como fonte única de despesa no Fluxo de Caixa, com comentário explicando o cuidado de não somar compras/contas em dobro.
- Antecipação de recebíveis marca `is_anticipated` e o `receive_installment` pula o lançamento para não duplicar o fluxo de caixa.
- Taxas de cartão no DRE são calculadas ao vivo (`payments × fee%`), cobrindo D+0 e D+N.
- Recorrência de contas, roteamento de receita para bancos, conciliação e aging — escopo de um ERP financeiro sério.

BUG-39 — Receita em três fontes divergentes; telas financeiras não reconciliam

- **Severidade:** CRÍTICA (confiabilidade) · **Módulo:** Financeiro · **Tipo:** Regra de negócio / Arquitetura
- **Evidência:** Visão Geral lê receita de `orders` ([VisaoGeralFinTab.tsx:90](#)); Fluxo de Caixa idem ([FluxoCaixaTab.tsx](#)); DRE-Caixa lê de `payments` ([DRETab.tsx:228](#)); DRE-Competência soma `payments + fin_receivable_installments` ([DRETab.tsx:738](#)). Não há um livro-razão único.
- **Impacto:** O faturamento do mesmo período difere conforme a aba. O gestor não consegue confiar em nenhum número isoladamente.
- **Sugestão:** Adotar `fin_cash_flow` (ou um ledger dedicado) como **fonte única** de receita e despesa, e fazer todas as telas lerem dele; manter `orders/payments` apenas como detalhamento.

BUG-40 — Visão Geral e Fluxo de Caixa misturam regimes e contam pedido não pago como receita

- **Severidade:** ALTA · **Módulo:** Financeiro · **Tipo:** Regra de negócio
- **Evidência:** Em [VisaoGeralFinTab.tsx:90-117](#), a **receita** vem de `orders.total_amount` (regime de competência, no momento da venda, **sem filtrar `is_paid`**) enquanto a **despesa** vem de `fin_cash_flow` (regime de caixa, quando paga).
- **Impacto:** "Lucro estimado" mistura receita de competência com despesa de caixa — uma compra a prazo gera "lucro" sem custo até ser paga. Além disso, pedidos não pagos (mesa aberta, fiado) entram como receita já realizada.
- **Sugestão:** Escolher um regime e aplicá-lo dos dois lados; filtrar receita por `is_paid` (caixa) ou reconhecer custo no mesmo regime (competência).

BUG-41 — DRE Competência duplica vendas a prazo (pagamento + recebível)

- **Severidade:** ALTA · **Módulo:** Financeiro/DRE · **Tipo:** Regra de negócio
- **Evidência:** [DRETab.tsx:738-739](#): `receitaBruta = receitaRecebida + receitaARceber`, onde `receitaRecebida` soma `payments` (que já inclui o cartão a prazo, pois `record_payment` cria a linha em `payments` no momento da venda) e `receitaARceber` soma `fin_receiveable_installments` pendentes (que representam **essas mesmas vendas**).
- **Reprodução:** Venda no cartão a prazo (D+30) → no acumulado/ano, aparece em `payments` (mês da venda) e em recebíveis (mês do vencimento) → contada duas vezes.
- **Impacto:** Receita bruta e margem do DRE-Competência infladas para qualquer operação com cartão parcelado/a prazo.
- **Sugestão:** No regime de competência, a receita já está em `payments` (momento da venda); o "a receber" é saldo, não receita adicional — não somar.

BUG-42 — DRE "Caixa" inclui cartão a prazo ainda não recebido

- **Severidade:** MÉDIA · **Módulo:** Financeiro/DRE · **Tipo:** Regra de negócio
- **Evidência:** [DRETab.tsx:228-236](#) — o regime "Caixa" soma `payments.amount` por `created_at`, incluindo pagamentos em cartão com `days_to_receive > 0` que ainda não entraram em conta.
- **Impacto:** O "caixa" do período superestima o dinheiro efetivamente disponível.
- **Sugestão:** No regime de caixa, reconhecer cartão a prazo só na data de recebimento (recebível liquidado), não na venda.

BUG-43 — Taxa de cartão de recebíveis a prazo não entra no fluxo de caixa; recebimento lança receita bruta

- **Severidade:** MÉDIA · **Módulo:** Financeiro · **Tipo:** Regra de negócio
- **Evidência:** Em `order-write.record_payment`, a despesa de taxa (`auto_card_fee`) só é lançada no ramo `days_to_receive === 0` ([order-write/index.ts:876-884](#)). Para cartão a prazo, `receive_installment` lança `fin_cash_flow` income pelo **valor cheio** da parcela, sem a taxa ([financial-write/index.ts:517](#)).
- **Impacto:** No Fluxo de Caixa/Visão Geral (que leem `fin_cash_flow`), as taxas de maquininha de vendas a prazo desaparecem → despesa subestimada e lucro superestimado. (O DRE não sofre porque recalcula a taxa ao vivo — gerando mais uma divergência entre telas.)
- **Sugestão:** Lançar a taxa também para recebíveis (no recebimento), e registrar o income líquido ou income bruto + taxa.

BUG-44 — Receita do DRE não exclui pagamentos estornados (`is_refunded`)

- **Severidade:** MÉDIA · **Módulo:** Financeiro/DRE · **Tipo:** Regra de negócio
- **Evidência:** As queries de `payments` do DRE ([DRETab.tsx:73-81](#) e [228-236](#)) filtram por status do pedido (`!= cancelled`) mas **não filtram `is_refunded`**. Um pagamento estornado sem cancelar o pedido (ou estorno parcial) continua somando como receita.
- **Impacto:** Receita do DRE superestimada quando há estorno sem cancelamento do pedido.
- **Sugestão:** Adicionar `.eq('is_refunded', false)` nas leituras de `payments` para receita.

Atualização da matriz de riscos (Seção 11)

Risco	
Alto	Risco de perda de receita devido a falhas no sistema de pagamento.
Médio	Risco de perda de receita devido a falhas no sistema de pagamento.
Baixo	Risco de perda de receita devido a falhas no sistema de pagamento.

Veredito do módulo financeiro: é o módulo mais **ambicioso** do sistema e, em funcionalidades, está à frente de muitos concorrentes. Mas, na **ótica financeira**, ainda não é confiável: faltam um livro-razão único e a disciplina de um regime contábil consistente. Hoje, quatro telas dão quatro faturamentos diferentes para o mesmo período — o que, num módulo financeiro, é o defeito mais grave possível, porque mina a confiança em todos os números. A base de dados (**fin_cash_flow**, recebíveis, pagáveis, bancos) já existe para virar esse ledger único; o trabalho é de **consolidação e consistência de regime**, não de construir do zero.

Auditoria somente-leitura. Nenhuma correção foi implementada, conforme solicitado.